

Electrical & Computer Project Repository Hub

Technical Documentation and Deployment Guide

Version 2.5.0 Nightly Build

Domain: Electrical & Electronics Engineering project submissions
Frontend: React, Vite, Tailwind CSS
Backend: Node.js, Express.js
Storage: Official GitHub project repository
Workflow: Upload branch, pull request, auto-merge, auto-pull
Prepared By: Vaishnav Ceh
Creator Profile: github.com/vaishnavceh

June 11, 2026

Contents

1	Executive Summary	2
1.1	Current Version	2
2	Version 2.5.0 Nightly Update	2
2.1	Issues Solved	2
2.2	New Updates	3
3	Purpose and Scope	3
3.1	Purpose	3
3.2	Scope	3
4	Repository Strategy	4
5	System Architecture	4
5.1	Architecture Overview	5
5.2	Request Flow	5
6	Technology Stack	5
7	Frontend Pages	6
7.1	Home	6
7.2	Documentation	6
7.3	Upload Project	6
7.4	Files	8
7.5	Guidelines	8
7.6	Repository	8
7.7	Submission Guide	8
7.8	Templates and Resources	9
7.9	Know More	9
8	Admin Pane	9

8.1	Admin Features	9
8.2	Passkey Configuration	10
9	Upload Validation Rules	10
9.1	Required Fields	10
9.2	Team Number Rule	11
9.3	Blocked Uploads	11
10	Generated README.md	11
11	Overwrite Behavior	12
12	GitHub Actions Auto-Merge Workflow	12
12.1	Workflow Settings	13
13	Environment Variables	13
13.1	Frontend	14
13.2	Backend	14
14	Deployment Guide	14
14.1	Frontend Deployment	14
14.2	Backend Deployment	15
15	Testing Checklist	15
16	Security Notes	15
17	Troubleshooting	16
18	Future Enhancements	16
19	Conclusion	17

Executive Summary

The Electrical & Computer Project Repository Hub is a full-stack web platform for collecting, organizing, and publishing student project submissions. It replaces manual Git usage for students with a guided website upload flow. The backend receives project files, validates submission data, creates a GitHub upload branch, opens a pull request, and supports an automated workflow for merging accepted uploads.

The system is designed for department-level use. It separates website source code from student project storage, avoids exposing deployment endpoints on public pages, and keeps credentials only in backend environment variables.

Current Version

This document describes **version 2.5.0 nightly build**. This version improves the student submission workflow with searchable batch and semester dropdowns, automatic repository path formatting, guide and faculty metadata, default Electrical & Electronics Engineering department storage, stricter GitHub URL validation, and updated documentation.

Version 2.5.0 Nightly Update

Issues Solved

- Students select batch and semester instead of typing repository folder names.
- Batch values such as 2027 are stored as `batch-2027`.
- Semester values such as Semester 6 are stored as `semester-6`.
- Guide name is required and guide designation is optional.
- Faculty name and faculty designation are optional.
- Guide and faculty departments default to `Electrical & Electronics Engineering (EEE)`.
- GitHub repository links are validated before upload.
- Project title and subject values are converted into repository-safe folder names automatically.

New Updates

- Application version updated to 2.5.0 nightly build.
- Upload form includes searchable batch and semester dropdowns.
- Generated README files include project guide, faculty, department, and GitHub repository metadata.
- Documentation now includes the updated submission workflow, wiki page, user manual, and repository structure guidance.
- Validation messages are clearer for missing metadata, invalid GitHub URLs, and malformed repository paths.

Purpose and Scope

Purpose

The purpose of this website is to make student project submission structured, repeatable, and easier to maintain. Students should not need to clone repositories, create branches, or open pull requests manually. The website handles the submission flow and lets GitHub workflows manage accepted project updates.

Scope

The platform supports:

- Team-based project submissions.
- Searchable batch and semester selection.
- Project files and optional report PDF uploads.
- Required project guide metadata.
- Optional faculty attribution metadata.
- Default Electrical & Electronics Engineering department metadata.
- Official course, seminar, lab, presentation, and review template access.
- Optional report and working-video links.
- Optional README comments.

- GitHub pull request creation.
- Auto-merge workflow support for upload branches.
- Repository browsing inside the website.
- Admin-only status checks.

The platform does not act as a complete learning management system. Authentication, role-based student accounts, grading, and analytics are future enhancements.

Repository Strategy

The project uses two separate repositories.

Repository	Purpose
ER_Project-Hub	Website source code, backend source code, frontend source code, documentation, Docker setup, and deployment configuration examples.
ELECTRICAL-AND-COMPUTER-PROJECT-REPOSITORY-OFFICIAL	Official repository for student project files.

This separation keeps the website application clean and prevents student submissions from being mixed with source code.

System Architecture

Architecture Overview

Layer	Responsibility
Frontend	React/Vite application, student upload form, repository browser, admin pane, status UI, dark/light mode.
Backend	Express API, upload validation, file handling, GitHub branch creation, pull request creation, repository file listing.
GitHub Repo	Stores accepted project files under batch, semester, subject, and team folders.
GitHub Actions	Auto-merge workflow for trusted upload branches after checks pass.
Deployment	Frontend and backend are deployed separately and connected using environment variables.

Request Flow

1. Student opens the frontend website.
2. Student fills the upload form and selects files.
3. Frontend sends form data to the configured backend service.
4. Backend validates fields, filenames, and file safety rules.
5. Backend creates an upload branch in the storage repository.
6. If the same project folder already exists, backend deletes the old project files on the upload branch.
7. Backend uploads project files and generated README content.
8. Backend opens a pull request against `main`.
9. GitHub Actions handles the auto-merge workflow for valid upload branches.
10. Accepted files become visible through the repository browser.

Technology Stack

Technology	Use
------------	-----

React	Builds the browser-based user interface.
Vite	Provides fast development and optimized production builds.
Tailwind CSS	Provides utility-based styling and responsive layout.
Lucide React	Provides icons for navigation, buttons, status panels, and controls.
Node.js	Runtime for the backend service.
Express.js	API routing and middleware.
Multer	Handles multipart project file uploads.
Octokit	Communicates with the GitHub REST API.
Docker Compose	Supports local development and testing.
Vercel	Intended frontend deployment platform.
Node Web Service	Intended backend deployment environment.

Frontend Pages

Home

The Home page summarizes the platform workflow. It tells users that project uploads are handled by the website, that auto-merge and auto-pull are enabled after checks pass, and that concerns should be reported to the admin. Documentation is available from its own navigation tab so the Home page does not automatically load the PDF.

Documentation

The Documentation page provides a click-to-load PDF.js preview of the latest compiled PDF documentation. The PDF is loaded only after the user requests the preview, which prevents the Home page from triggering an automatic browser download and avoids relying on blocked Google Docs embedding.

Upload Project

The Upload Project page is the main student submission page. It collects:

- Batch selected from a searchable dropdown, for example 2027, stored as `batch-2027`.
- Semester selected from a searchable dropdown, for example Semester 6, stored as `semester-6`.
- Subject or course area, stored as a repository-safe slug such as `dbms`.

- Team number from `team-01` through `team-100`.
- Project title, stored as a repository-safe slug such as `library-management`.
- Student information.
- GitHub repository link.
- Guide name, required.
- Guide designation, optional.
- Faculty name and designation, optional.
- Department, stored internally as `Electrical & Electronics Engineering (EEE)`.
- Project description.
- Tools used.
- Technologies used.
- Sources or references used.
- Problem statement, optional. If omitted, the README derives it from the project description.
- Installation or setup instructions, optional.
- Steps to run or test, optional.
- Screenshots, photos, or demo output notes, optional.
- Presentation details, optional.
- Future improvements, optional.
- Additional README comments, optional.
- Project files.
- Project report PDF, optional.
- Google Drive report link, optional.
- Working video link, optional.

After upload, the page shows the created branch, pull request link, project path, and Git process window. If the upload replaces an existing project folder, the success panel reports how many existing files were replaced.

Files

The Files page lets users browse accepted project files from the official storage repository inside the website. It does not expose GitHub tokens or backend secrets.

Guidelines

The Guidelines page explains upload expectations. It clarifies that students should submit through the website and that normal uploads are handled by the automated workflow after checks pass.

Repository

The Repository page explains the required storage folder structure:

```
batches/{batch}/{semester}/{subject}/{team-number}-{project-name}/
```

Standard batch layout example:

```
batch-2027/  
|-- semester-1/  
|-- semester-2/  
|-- semester-3/  
|-- semester-4/  
|-- semester-5/  
|-- semester-6/  
|-- semester-7/  
'-- semester-8/
```

Example:

```
batches/batch-2027/semester-6/dbms/team-01-library-management/
```

Submission Guide

The Submission Guide combines the previous README, Hardware, and Rules pages into one minimal checklist. It explains what a useful generated README should contain, what hardware evidence is useful for electronics and electrical projects, and which safety rules protect the repository. Students are reminded not to upload secrets, tokens, passwords, private keys, unrelated private data, or unsafe executable files.

Templates and Resources

This page shows the actual files and folders from the live official **TEMPLATES** directory in the storage repository. Students can open template folders, view template file names, open files on GitHub, and download available files before preparing their submissions.

Official templates repository:

```
https://github.com/ELECTRICAL-AND-COMPUTER-TKMCE/ELECTRICAL-AND-  
-COMPUTER-PROJECT-REPOSITORY-OFFICIAL
```

Official templates folder:

```
https://github.com/ELECTRICAL-AND-COMPUTER-TKMCE/ELECTRICAL-AND-  
-COMPUTER-PROJECT-REPOSITORY-OFFICIAL/tree/main/TEMPLATES
```

Know More

The Know More page shows version information, workflow status, solved issues, operational notes, and planned features. It also includes the creator GitHub profile:

```
https://github.com/vaishnavceh
```

Admin Pane

The Admin Pane is located in the lower area of the Home page. It is protected by passkeys configured through frontend environment variables.

Admin Features

- Frontend status check.
- Backend/project service status check.
- Runtime configuration check.
- Repository access check.
- Expandable command/process log.

Passkey Configuration

The frontend reads the admin passkey from:

```
VITE_ADMIN_PASSKEY=<admin-passkey>
```

Multiple passkeys can be supported with:

```
VITE_ADMIN_PASSKEYS=<passkey-one>,<passkey-two>
```

Upload Validation Rules

Required Fields

- Project title.
- Batch.
- Semester.
- Subject.
- Team number.
- Student information.
- GitHub repository link.
- Guide name.
- Project description.
- Tools used.
- Technologies used.
- Sources or references used.
- Project type.
- Confirmation checkbox.

Team Number Rule

Batch values are selected as years such as 2027 and stored as `batch-2027`. Semester values are selected as `Semester 1` through `Semester 8` and stored as `semester-1` through `semester-8`.

The GitHub repository link must use a valid `github.com/{owner}/{repo}` URL. Guide and faculty department metadata is stored as `Electrical & Electronics Engineering (EEE)` for version 2.5.0.

Valid examples:

```
team-01
team-09
team-10
team-99
team-100
```

Invalid examples:

```
team-00
team-101
team-1
team-001
```

Blocked Uploads

The backend blocks filenames that appear unsafe, including executable files, private key files, environment files, token files, credential files, and path traversal attempts.

Generated README.md

For each project, the backend creates a starter `README.md`. It includes:

- Project title.
- Student information.
- Batch, semester, subject, project type, and team folder.
- GitHub repository link.
- Guide name, guide designation, and guide department.

- Faculty name, faculty designation, and faculty department.
- Project description.
- Tools used.
- Technologies used.
- Sources or references.
- Project file location.
- Report PDF status.
- Optional report link.
- Optional working video link.
- Optional additional comments.
- `Not available.` for optional sections that are not provided and cannot be read from the report PDF.

Overwrite Behavior

When a team uploads a project to the same batch, semester, subject, team number, and project name path, the backend no longer rejects it as a duplicate. Instead, it creates an upload branch from the latest `main`, removes the existing files in that project folder on the upload branch, uploads the new files, and opens a pull request.

This means the replacement is still tracked by Git history. After the pull request is auto-merged, users can run `git pull origin main` in the storage repository to receive the overwritten project folder.

GitHub Actions Auto-Merge Workflow

The Auto Merge workflow must be created in the official storage repository, not in the website source repository.

File path:

```
.github/workflows/auto-merge-student-uploads.yml
```

Workflow:

```
name: Auto merge student uploads

on:
  pull_request:
    types: [opened, synchronize, reopened, ready_for_review]
    branches:
      - main

permissions:
  contents: write
  pull-requests: write

jobs:
  auto-merge:
    if: startsWith(github.head_ref, 'upload/')
    runs-on: ubuntu-latest

    steps:
      - name: Auto merge upload pull request
        run: gh pr merge "$PR_URL" --squash --auto --delete-branch
        env:
          GH_TOKEN: ${ secrets.GITHUB_TOKEN }
          PR_URL: ${ github.event.pull_request.html_url }
```

Workflow Settings

In the storage repository, configure:

- **Settings** → **Actions** → **General**.
- Enable **Read and write permissions**.
- Enable **Allow GitHub Actions to create and approve pull requests**.

The workflow only applies to branches that start with `upload/`. Normal manual pull requests are not auto-merged by this rule.

Environment Variables

Frontend

```
VITE_API_BASE_URL=<configured-project-service-url>
VITE_ADMIN_PASSKEY=<admin-passkey>
VITE_STORAGE_REPOSITORY_URL=https://github.com/ELECTRICAL-AND-COMPUTER-TKMCE/ELECTRICAL-AND-COMPUTER-PROJECT-REPOSITORY-OFFICIAL.git
```

Note: The deployed project service URL should be configured in Vercel or local environment files. It should not be hardcoded into committed source files.

Backend

```
GITHUB_OWNER=ELECTRICAL-AND-COMPUTER-TKMCE
GITHUB_REPO=ELECTRICAL-AND-COMPUTER-PROJECT-REPOSITORY-OFFICIAL
GITHUB_BASE_BRANCH=main
GITHUB_TOKEN=<server-side-token>
FRONTEND_URL=<frontend-origin>
CORS_ORIGIN=<frontend-origin>
NODE_ENV=production
PORT=10000
```

Important: The GitHub token must never be placed in frontend code, public screenshots, documentation, or Git history.

Deployment Guide

Frontend Deployment

The frontend can be deployed to Vercel.

1. Import the website source repository into Vercel.
2. Set the frontend root directory if required.
3. Add the frontend environment variables.
4. Deploy the frontend.
5. Copy the final frontend origin.
6. Update backend CORS settings with the final frontend origin.

Backend Deployment

The backend should run as a Node web service.

```
npm install
npm start
```

Backend health check path:

```
/api/health
```

Routes such as `/` or `/api` may return **Cannot GET** if not defined. That is acceptable when `/api/health` returns a valid health response.

Testing Checklist

No.	Test Item	Status
1	Frontend loads successfully.	Pending
2	Light mode and dark mode work correctly.	Pending
3	Admin Pane login works with configured passkey.	Pending
4	Backend health status check works from Admin Pane.	Pending
5	Batch dropdown stores 2027 as <code>batch-2027</code> .	Pending
6	Semester dropdown stores Semester 6 as <code>semester-6</code> .	Pending
7	Guide name and GitHub repository link are required.	Pending
8	Upload form accepts <code>team-01</code> through <code>team-100</code> .	Pending
9	Upload form rejects invalid team numbers.	Pending
10	Generated <code>README.md</code> includes guide, faculty, department, and GitHub repository metadata.	Pending
11	Backend creates an upload branch in the official storage repository.	Pending
12	Backend opens a pull request against <code>main</code> .	Pending
13	GitHub Actions auto-merge workflow runs for <code>upload/</code> branches.	Pending
14	Accepted project files appear in the repository browser.	Pending

Security Notes

- Store GitHub tokens only in backend environment variables.
- Never commit `.env` files.

- Rotate any token that was shown in a screenshot, browser page, terminal, or chat.
- Use the minimum GitHub permissions required for contents and pull requests.
- Prefer a department bot account or GitHub App for long-term deployment.
- Keep backend deployment endpoints out of public website pages.
- Do not allow students to submit passwords, API keys, private keys, or secret files.

Troubleshooting

Problem	Likely Fix
Frontend says backend is not reachable	Check frontend environment variable <code>VITE_API_BASE_URL</code> , backend CORS origin, and backend service status.
Upload fails with team number error	Use <code>team-01</code> through <code>team-100</code> .
Pull request is created but not merged	Check GitHub Actions permissions and confirm the branch starts with <code>upload/</code> .
Repository browser does not show files	Confirm backend environment points to the official storage repository and token has access.
Admin Pane login fails	Confirm <code>VITE_ADMIN_PASKEY</code> or <code>VITE_ADMIN_PASKEYS</code> is configured and redeploy frontend.

Future Enhancements

- Student login and team submission history.
- Admin dashboard for accepted uploads and rejected submissions.
- Search and filters by batch, semester, subject, team, and technology.
- More detailed GitHub Actions status inside the website.
- Course material templates, seminar templates, report templates, and lab resources.
- Department-level project analytics.
- Additional department options such as Computer Engineering, Electronics & Communication Engineering, Mechanical Engineering, and Civil Engineering.

Conclusion

The Electrical & Computer Project Repository Hub provides a structured and safer workflow for collecting department project submissions. Version 2.5.0 nightly build simplifies student submissions through batch and semester dropdowns, automatic repository path formatting, required guide metadata, optional faculty metadata, default Electrical & Electronics Engineering department storage, stronger validation, and updated documentation. The system is prepared for frontend hosting, backend production use, and official storage repository testing.